

主要内容

- 图灵机简介、形式化定义
- 图灵机的表示能力：语言、停机与整数运算
- 扩展图灵机
- 受限图灵机

图灵机的语言

- 定义: 如果 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ 是一个图灵机, 则 M 接受的语言为
$$L(M) = \{w \mid w \in \Sigma^*, q_0 w \vdash^* \alpha p \beta, p \in F, \alpha, \beta \in \Gamma^*\}$$
 - 输入串放在输入带上, M 处于 q_0 , 带头位于输入串的第一个字符上, 输入串最终会导致 M 进入某个终止 (接受) 状态。
- 一般假定当输入串被接受时 M 总会停机 (halt), 即没有下一个动作的定义。
- 而对于不接受的输入, TM 可能永远不停机。
 - 无法知道, 到底是因为运行的时间不够长而没有接受, 还是根本就不会停机。

- 定义: 如果 L 是图灵机 M 的语言, 则称 L 是**递归可枚举** (Recursively Enumerable) 语言。
 - “递归”: 一个在计算机出现之前的数学术语
 - 与现在熟悉的递归函数不一样, 术语“递归”是“可判定的”同义词
 - “可枚举”: 语言中的串可以被某个图灵机枚举出来。
 - “递归可枚举”: 按照某种顺序列举出来的语言

递归语言

- 递归语言 (**recursive languages**)

称语言 L 是递归的，当且仅当存在图灵机 M ，使得 $L = L(M)$ ，且满足：

1. 如果 $w \in L(M)$ ，则对于 w ， M 接受 w （自然会停机，到达终态）
2. 如果 $w \notin L(M)$ ，则对于 w ， M 最终也会停机（虽然不能到达终态）

- 算法的形式化

保证停机的图灵机，正是算法的好模型，这也是算法概念的首次形式化，并由此建立了计算机科学的基础。

整数函数计算器

- 整数函数计算器：图灵机可以作为语言的识别器，也可以用作整数函数计算器。

- 例如，将整数 $i \geq 0$ 表示为字符串 0^i

- 若计算 k 个自变量 $i_1, i_2, \dots, i_k$ 的函数 f ，用
 $0^{i_1} 1 0^{i_2} \dots 1 0^{i_k}$

作为图灵机 M 的输入

- 当 M 停机且输入带上为 0^m ，表示计算

$$f(i_1, i_2, \dots, i_k) = m$$

整数函数计算器

- 例：设计整数真减法的图灵机

$$m \dot{-} n = \begin{cases} m - n, m \geq n \\ 0, m < n \end{cases}$$

- 思路：

- 开始时，带上符号为 $0^m 1 0^n$ ，结束时，带上的符号为 0 ($m \geq n$) 或者空格符 ($m < n$)。
- 每当在1的左边将一个0改变为B，就在1的右边将一个0改为1。重复以上循环，直到以下两种情况结束：
 - (1) 1右边的n个0全部改为了1，向右移遇到了空格符，此时把 (n+1) 个1改为1个0和n个B。这样带上剩下m-n个0。
 - (2) 向左已经找不到0，此时把带上剩下的1和0全部变为空格符号，留下一个只有空格符的带。

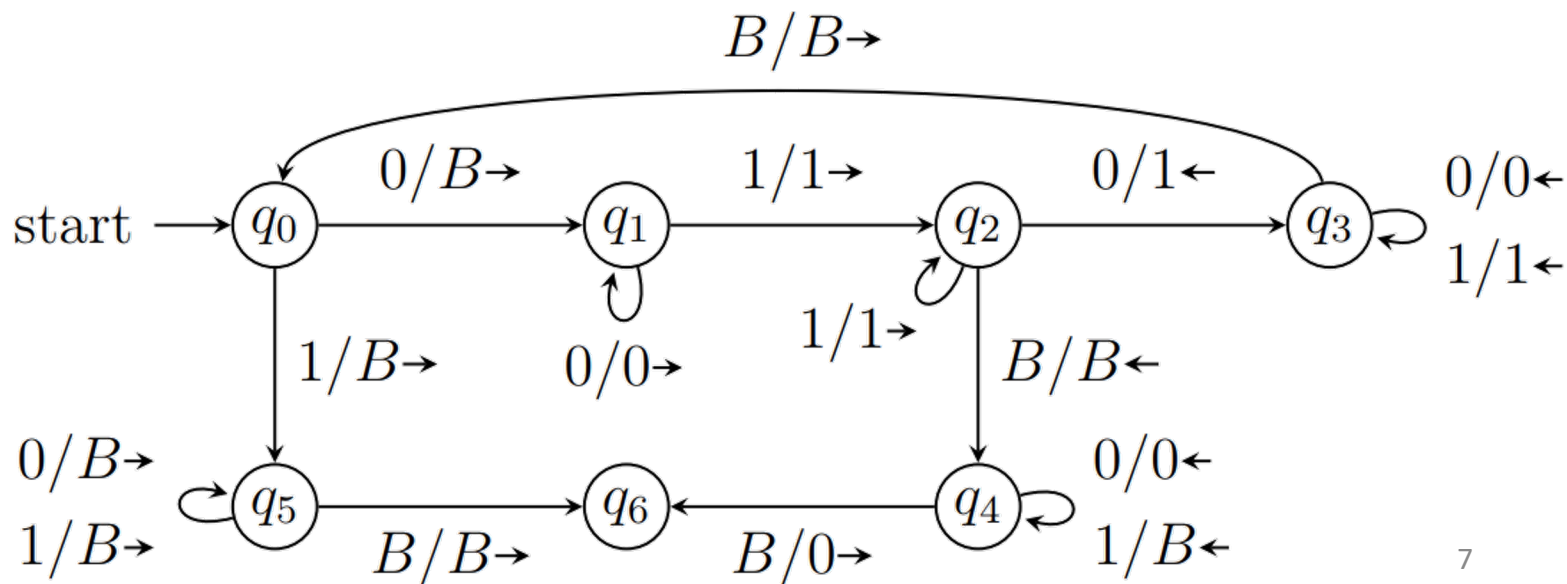
整数函数计算器

每当在1的左边将一个0改变为B，就在1的右边将一个0改为1。重复以上循环，直到以下两种情况结束：

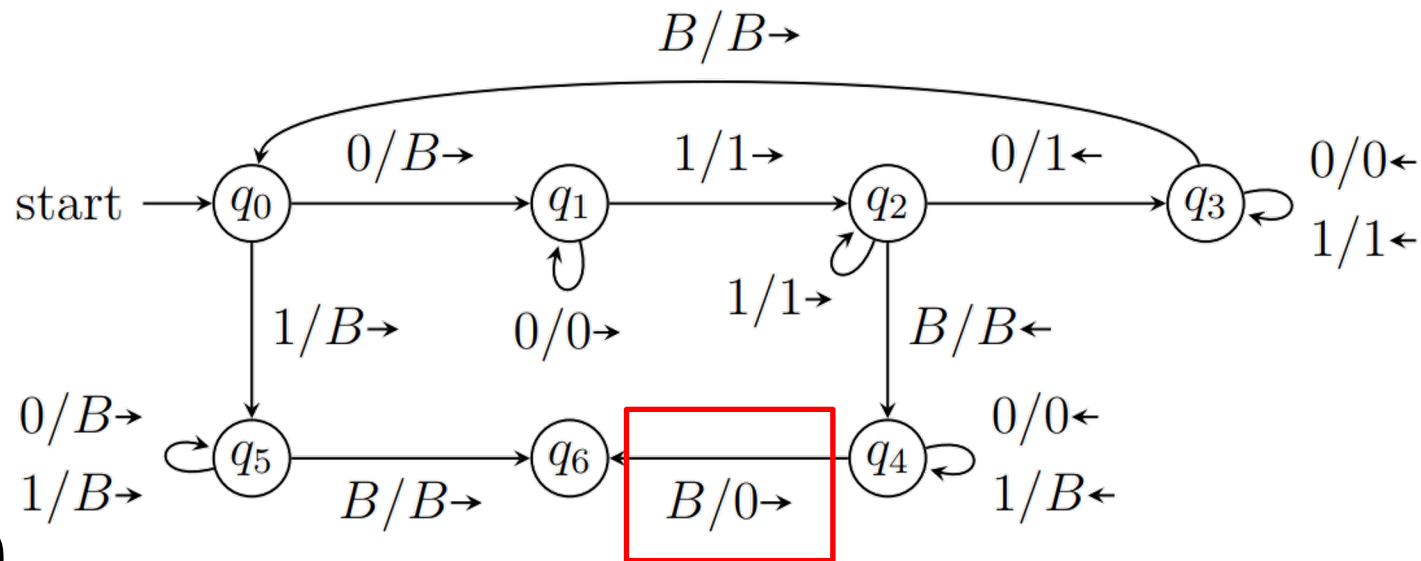
(1) 1右边的n个0全部改为了1，向右移遇到了空格符，此时把(n+1)个1改为1个0和n个B。这样带上剩下m-n个0。

(2) 向左已经找不到0，此时把带上剩下的1和0全部变为空格符号，留下一个只有空格符的带。

$$m \div n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$



红框中转移弧的作用



A

替换左部的一个B为0

B

减少一个0的计数

C

替换右部的一个B为0

D

增加一个0的计数

提交

整数函数计算器

例：二进制数的加1函数，用符号\$作为数前的占位标记。例如 $q_0\$10011 \vdash^* q_f 10100$ ， $q_0\$111 \vdash^* q_f 1000$

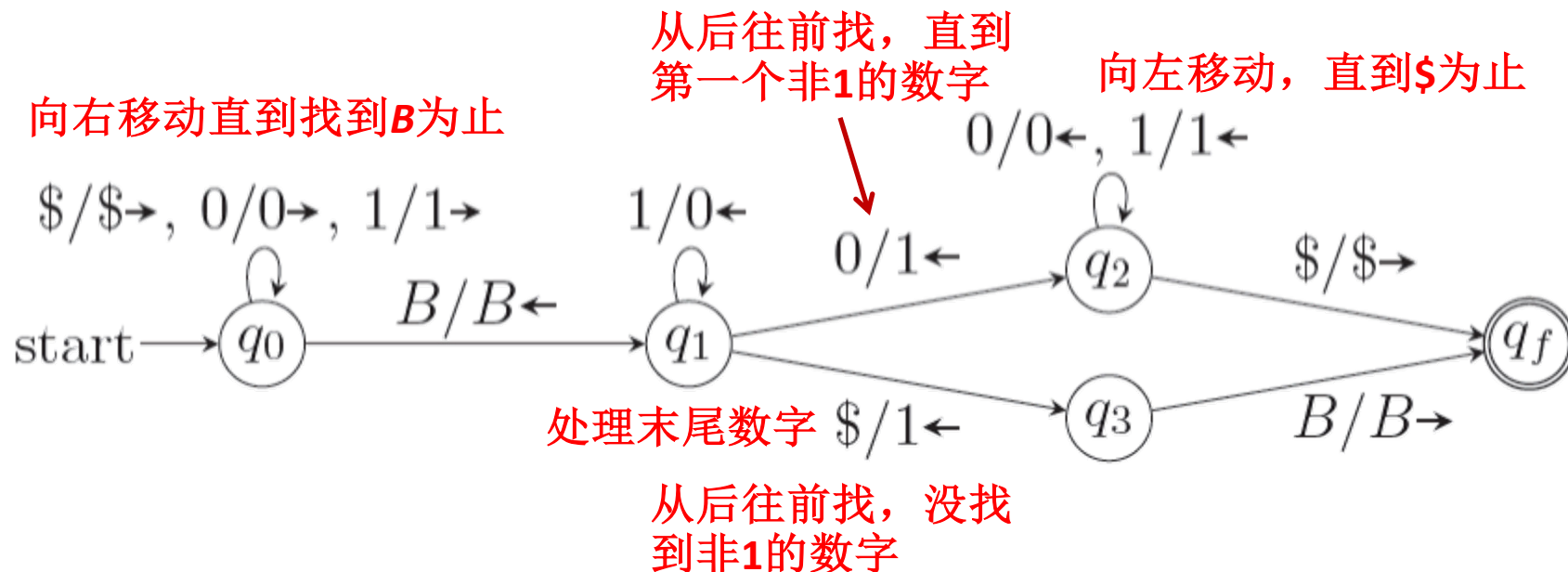
分两种情况：(1)末尾为1时，1变成0，往回直到遇到0或者\$，把0或者\$变成1；(2)末尾为0时，0变成1。第(2)种情况被第(1)种情况包含

整数函数计算器

例：二进制数的加1函数，用符号\$作为数前的占位标记。例如 $q_0\$10011 \vdash^* q_f 10100$, $q_0\$111 \vdash^* q_f 1000$

分两种情况：(1)末尾为1时，1变成0，往回直到遇到0或者\$，把0或者\$变成1；(2)末尾为0时，0变成1。第(2)种情况被第(1)种情况包含

$$M = (\{q_0, q_1, q_2, q_3, q_f\}, \{0, 1\}, \{0, 1, \$, B\}, \delta, q_0, B, \{q_f\})$$



主要内容

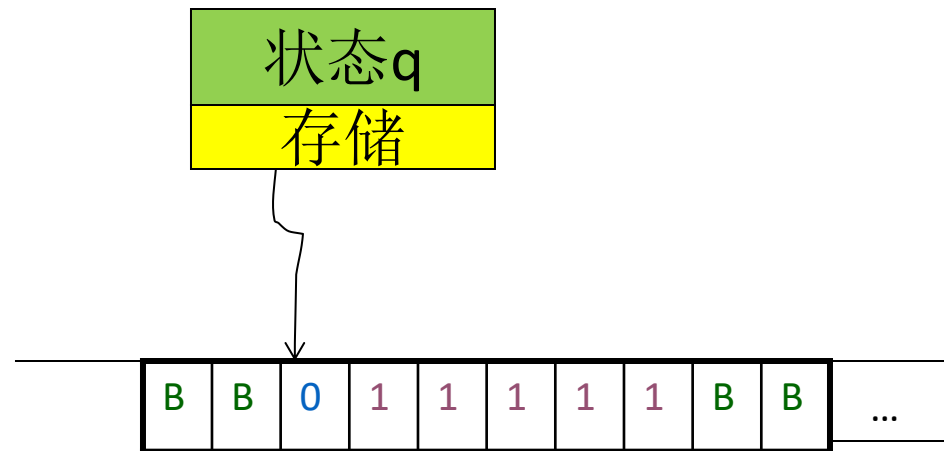
- 图灵机简介、形式化定义
- 图灵机的表示能力：语言、停机与整数运算
- 扩展图灵机
- 受限图灵机

带状态中存储的图灵机

- 状态中存储: 有穷控制器中可以存储有穷个符号的图灵机:

$$M' = (Q', \Sigma, \Gamma, \delta, q_0', B, F')$$

其中 $Q' = Q \times \Gamma \times \cdots \times \Gamma$, $q_0' = [q_0, B, \cdots, B]$ 。



图灵机的七元组形式没有改变，依然是基本的图灵机。

基本图灵机的几种编程技巧

例3: 设计一个带存储区的图灵机, 接受 $01^* + 10^*$ 。

基本图灵机的几种编程技巧

例3: 设计一个带存储区的图灵机, 接受 $01^* + 10^*$ 。

解: 该图灵机设计如下

$$M = (Q, \{0, 1\}, \{0, 1, B\}, \delta, [q_0, B], B, \{[q_1, B]\})$$

其中, 转移函数定义如下:

1. $\delta([q_0, B], 0) = ([q_1, 0], 0, R)$, $\delta([q_0, B], 1) = ([q_1, 1], 1, R)$
2. $\delta([q_1, 0], 1) = ([q_1, 0], 1, R)$, $\delta([q_1, 1], 0) = ([q_1, 1], 0, R)$
3. $\delta([q_1, 0], B) = ([q_1, B], B, R)$, $\delta([q_1, 1], B) = ([q_1, B], B, R)$

基本图灵机的几种编程技巧

例3: 设计一个带存储区的图灵机, 接受 $01^* + 10^*$ 。

解: 该图灵机设计如下

$$M = (Q, \{0, 1\}, \{0, 1, B\}, \delta, [q_0, B], B, \{[q_1, B]\})$$

其中, 转移函数定义如下:

1. $\delta([q_0, B], 0) = ([q_1, 0], 0, R)$, $\delta([q_0, B], 1) = ([q_1, 1], 1, R)$
2. $\delta([q_1, 0], 1) = ([q_1, 0], 1, R)$, $\delta([q_1, 1], 0) = ([q_1, 1], 0, R)$
3. $\delta([q_1, 0], B) = ([q_1, B], B, R)$, $\delta([q_1, 1], B) = ([q_1, B], B, R)$

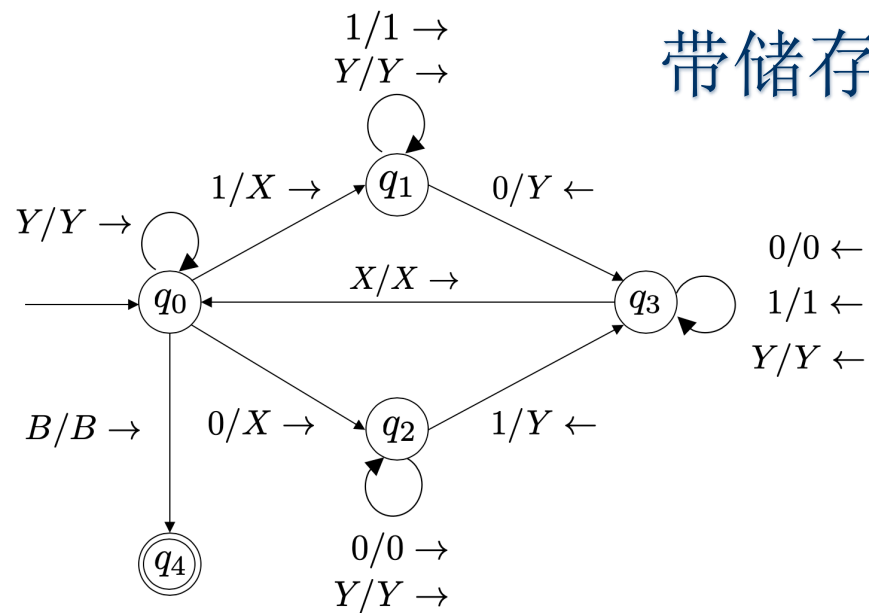
下面来分析一下例子100。

$$[q_0, B]100 \vdash 1[q_1, 1]00 \vdash 10[q_1, 1]0 \vdash 100[q_1, 1]B \vdash 100B[q_1, B]B$$

这里面, 数据存储的就是看到的第一个输入符号。其实还是一个基本的图灵机, 只不过每个状态是一个二元组。

设计图灵机

例2：为下列语言设计图灵机：带有相同个数的0和1的串的集合。



带储存区之后如何简化设计？

分析下一个例子1001:

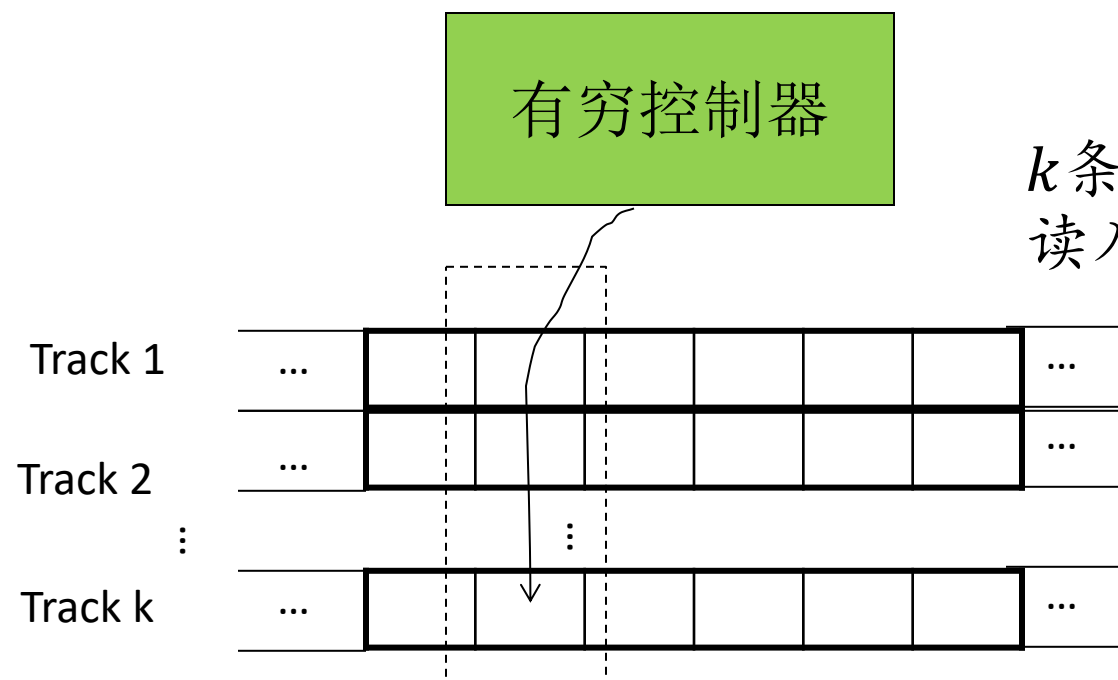
$q_0 1001 \vdash X q_1 001 \vdash q_3 XY01 \vdash X q_0 Y01 \vdash XY q_0 01 \vdash$
 $XY X q_2 1 \vdash XY q_3 XY \vdash XY X q_0 Y \vdash XY XY q_0 B \vdash$
 $XY XY B q_4 B$

多道图灵机

多道 (multi-track) 图灵机:

$$M' = (Q, \Sigma, \Gamma', \delta, q_0, B', F)$$

其中 $\Gamma' = \Gamma \times \Gamma \times \cdots \times \Gamma$ 。



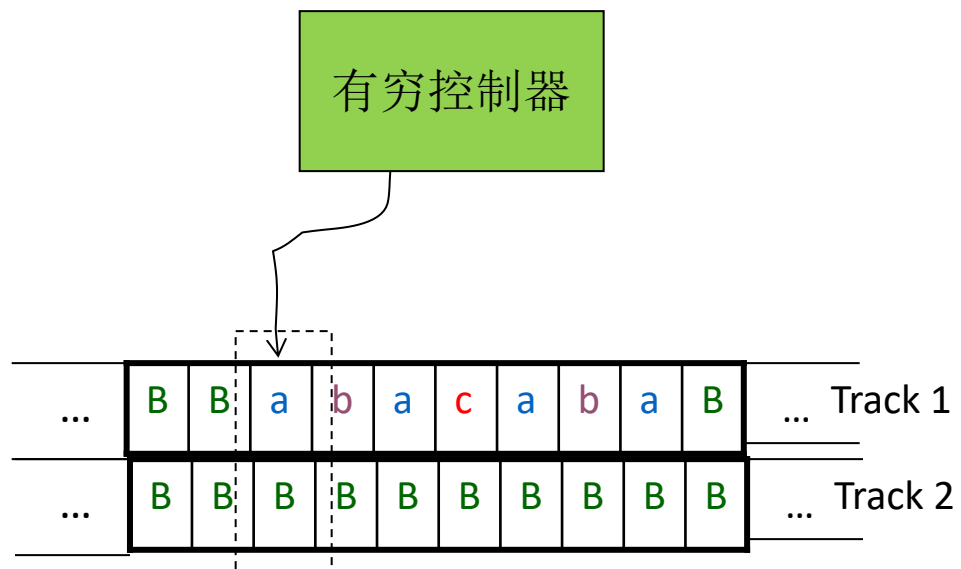
k 条道，读头一次
读入 k 个符号。

图灵机的七元组形式没有改变，依然是基本的图灵机。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

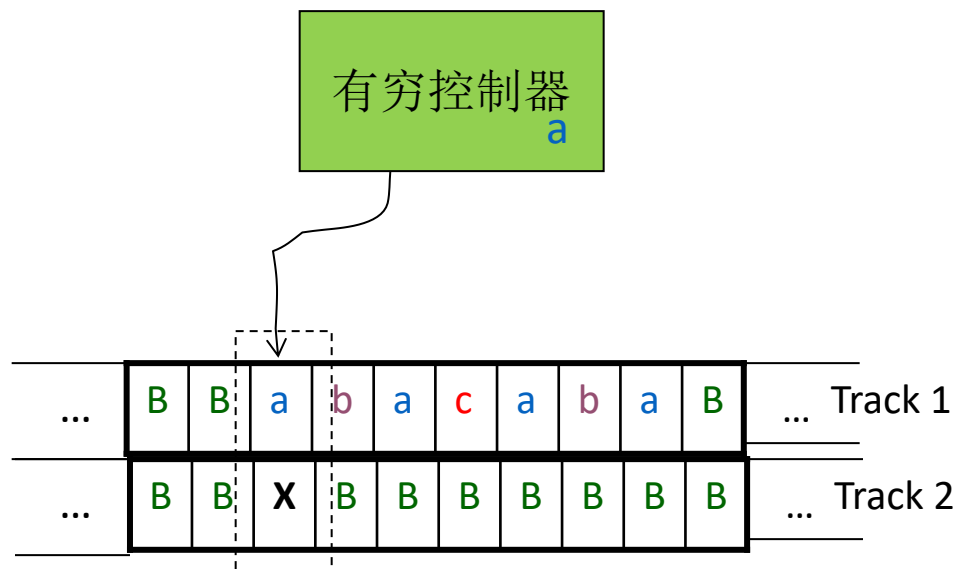


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

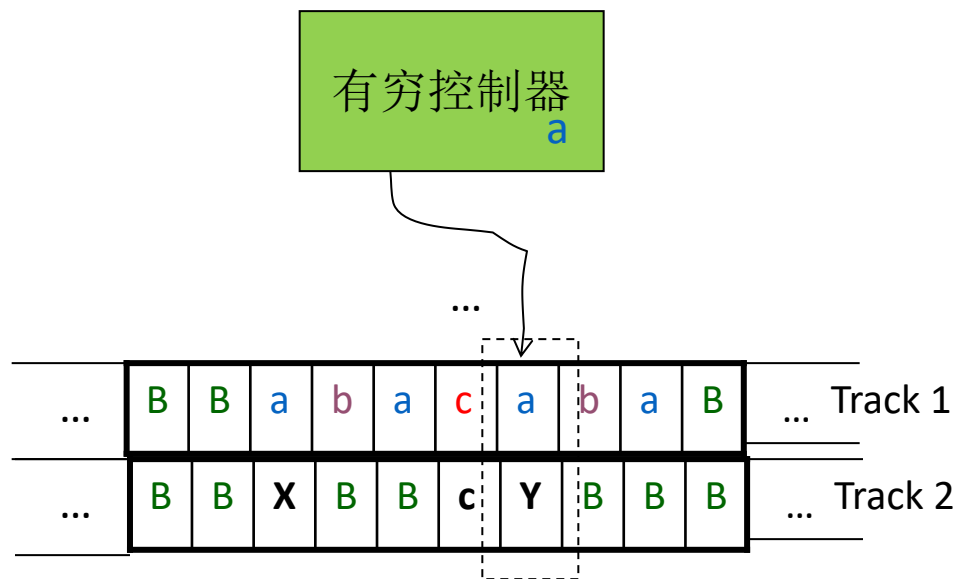


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

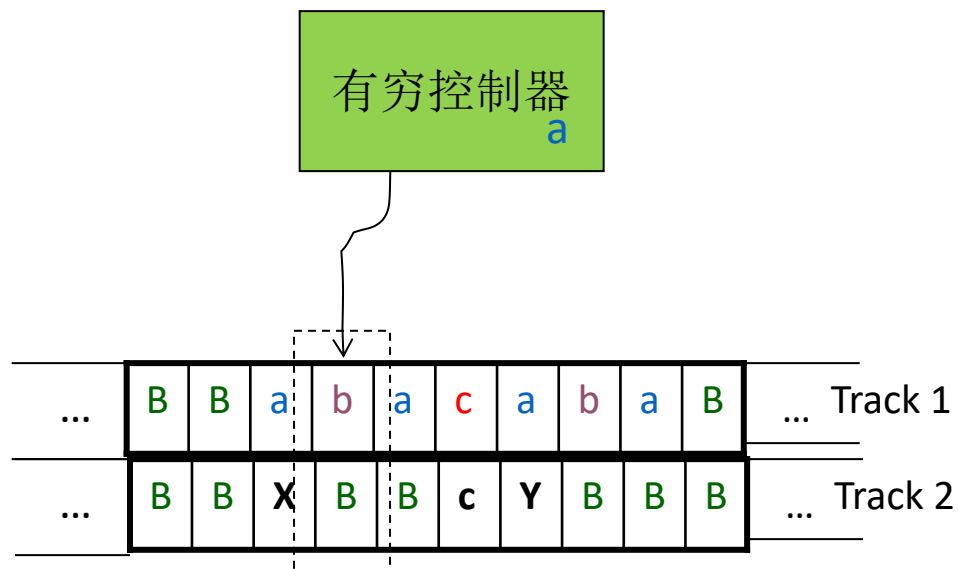


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

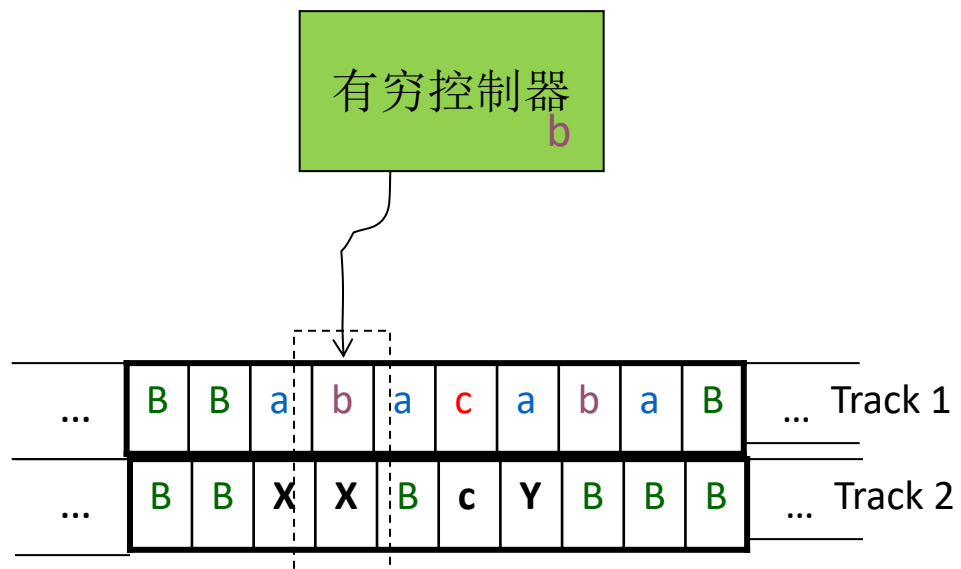


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

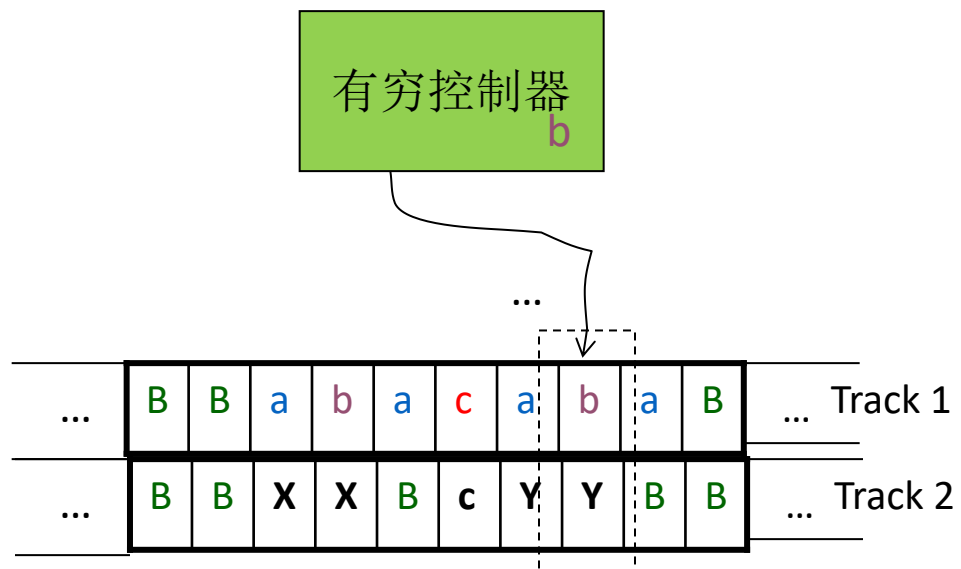


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

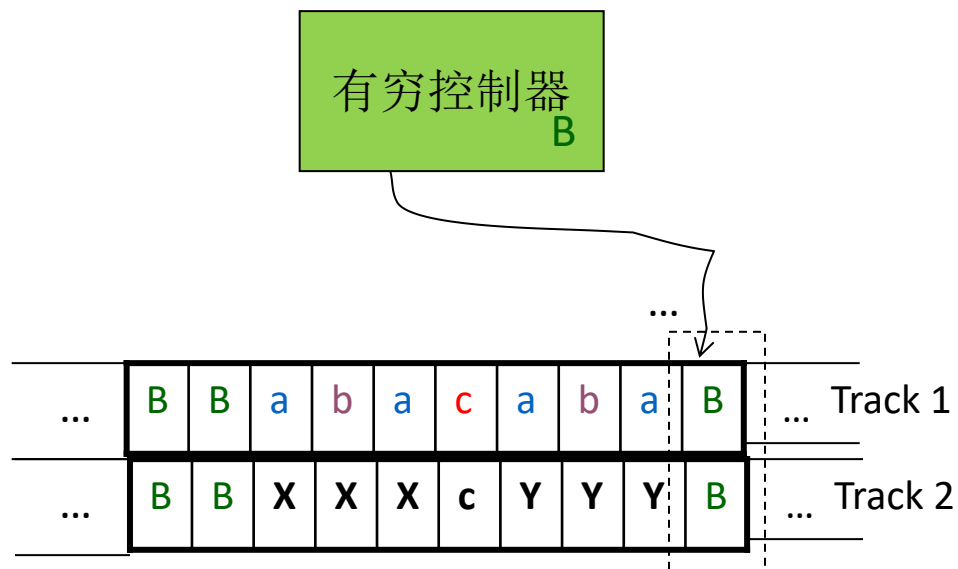


Track 2主要起到做标记的作用。

示例

- 例: 利用状态中存储与多道设计TM识别

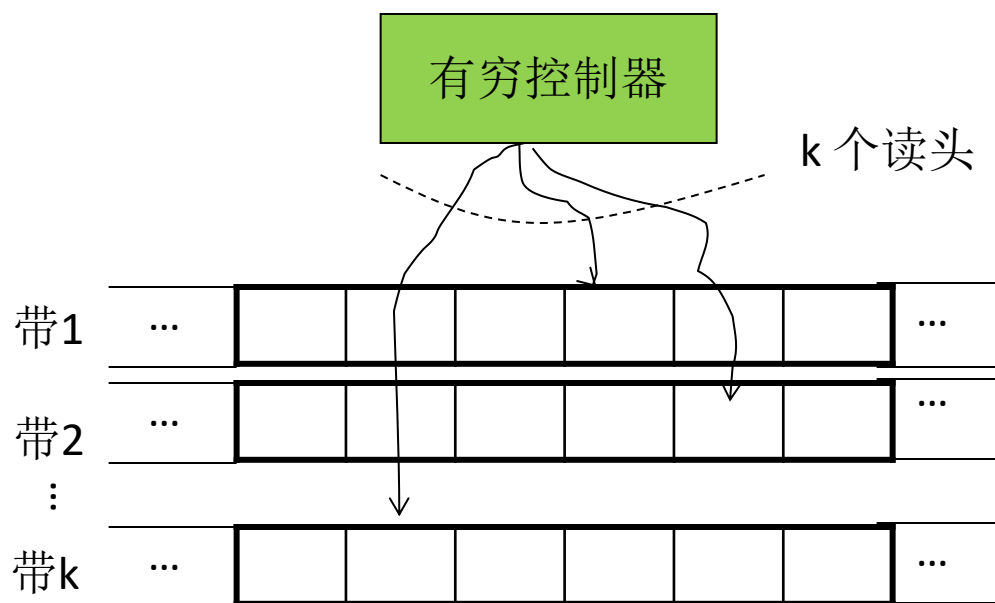
$$L = \{wcw | w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

多带图灵机

- 多带 (multi-tape) 图灵机：有穷控制器、 k 个带头和 k 条带组成。每个动作，根据状态和每个带头符号：
 - 改变控制器中的状态；
 - 修改每个带头单元格中的符号；
 - 每个带头独立的向左或右移动一个格，或保持不动。



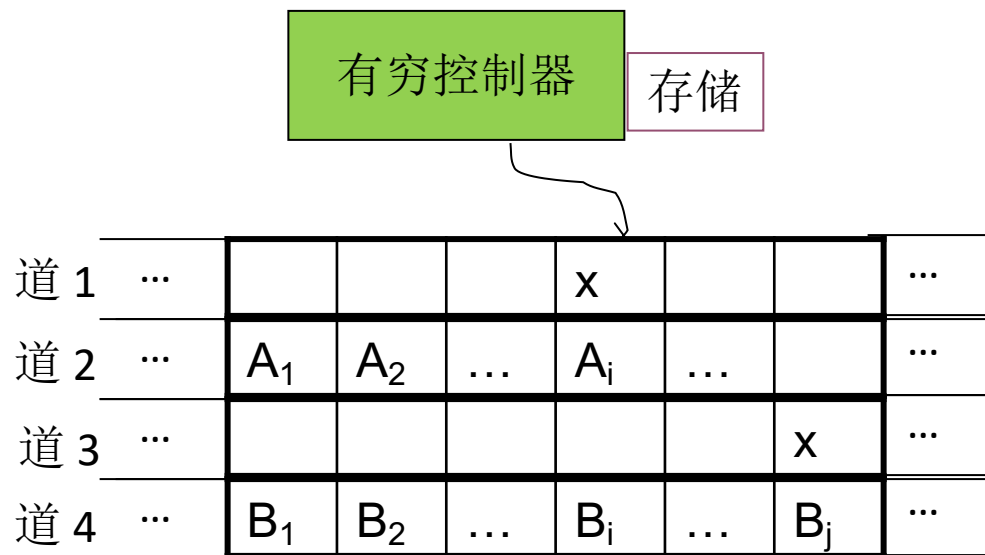
开始时，输入在第一条带上，其他带上都是空格符。

多带图灵机

- 定理: 多带图灵机接受的语言 L , 可被单带图灵机接受。

模拟多带图灵机:

- 用 $2k$ 道的单带图灵机 D 模拟 k 带图灵机 M ;
- D 用两道模拟 M 一带, 一道放置内容, 一道标记带头;
- 为模拟 M 的一个动作, D 需从左至右, 再从右至左, 各扫描一次;
- 第一次扫描收集各带头处符号, 第二次更新带头符号和位置。



⋮

- 图灵机在每组状态 q 和带符号 X 的转移 $\delta(q, X)$, 可以有有穷个选择:

$$\delta(q, X) = \{(q_1, Y_1, D_1), (q_2, Y_2, D_2), \dots, (q_k, Y_k, D_k)\}$$

(1) NTM接受语言的方式, 与NFA和PDA是类似的

(2) 只要存在从初始ID到某接受状态ID的转移, 其他选择可以忽略

非确定图灵机

- 定理：如果 L 被非确定图灵机 N 接受，那么 L 被某个确定的图灵机 M 接受。
- 证明要点：
 - 让等价的基本TM M 具有3条带。
 - 第1条带用来存放输入。
 - 第2条带上系统地生成 M 的各种可能的移动序列
 - M 在第3条带上按照第2条带上给出的移动系列处理输入串，如果成功，则接受之，如果不成功，则在第2条带上生成下一个可能的移动系列，开始新一轮的“试处理”。

主要内容

- 图灵机简介、形式化定义
- 图灵机的表示能力：语言、停机与整数运算
- 扩展图灵机
- 受限图灵机

半无穷带图灵机

- 半无穷带图灵机：图灵机输入输出带只有一侧是无穷的。
 - 定理：半无穷带图灵机，与图灵机等价。

半无穷带图灵机

- 证明方法：让一侧无穷的图灵机带使用多道技术，模拟双侧无穷的图灵机带。

双侧 无穷带	...	A_{-2}	A_{-1}	A_0	A_1	A_2	...
-----------	-----	----------	----------	-------	-------	-------	-----

半无穷带	A_0	A_1	A_2	...
	#	A_{-1}	A_{-2}	...

1. 当双侧无穷图灵机向右移动时，半无穷带图灵机也正常向右移动。

2. 当双侧无穷图灵机向左移动时，半无穷带图灵机正常也向左移动，需要越过左边边界位置的时候，通过特定的符号，让半无穷带图灵机进行跳转，向右移动，且使用另一条道即可。

✨思考如何实现？

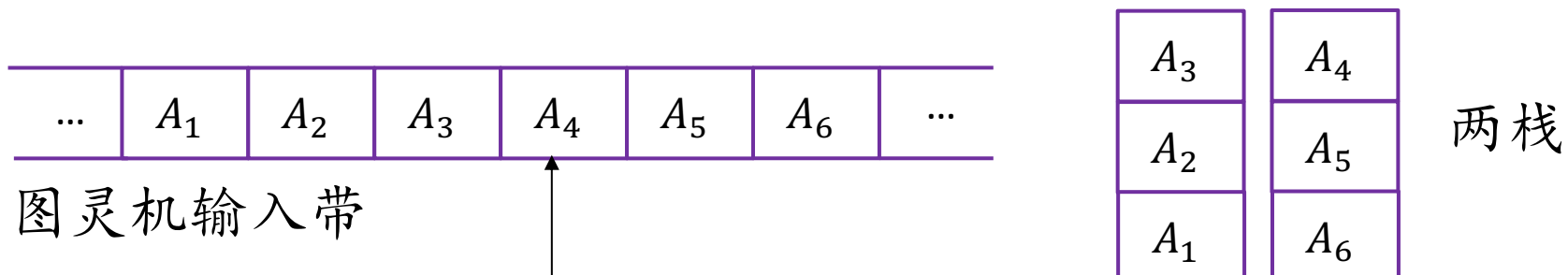
状态用二元组的形式 (q, L) 和 (q, R)

- 多栈机器：基于下推自动机的扩展， k 栈机器是具有 k 个栈的确定型下推自动机。
 - 具有多个下推栈的 PDA，包含 k 个下推栈的一步转移可表达为 $(p, \gamma_1, \gamma_2, \dots, \gamma_k) \in \delta(q, a, X_1, X_2, \dots, X_k)$.
 - 定理：如果图灵机接受 L ，那么双栈机接受 L 。

多栈机器

- 证明要点:

- 开始时输入在双栈机的输入带，可以先将输入扫描并压入一个栈，然后再依次把每个符号弹出再压入另一个栈，然后进入模拟图灵机的状态。
- 用两个堆栈模拟图灵机带，一个堆栈保存带头左边内容，一个堆栈保存带头右边内容，但都不包括无穷的空格符号。
- 带头的移动用两个栈分别弹栈和压栈模拟。若带头修改字符 A 为 B ，用一个栈弹出的字符 A 而另一个栈压入 B 来模拟。



- 示例：利用双栈机器接受 $L = \{a^n b^n c^n \mid n \geq 0\}$
 - 方案1
 - 栈A存所有的a，栈B存所有的b
 - 每个c同时和两个栈顶符号匹配
 - 方案2
 - 栈A存所有的a
 - 栈B存所有的b, 并匹配清空栈A中所有的a
 - 当栈A为空，基于栈B匹配c
 - 方案3
 - 完全模拟图灵机进行匹配

- 示例：利用双栈机器接受 $L = \{a^n b^n c^n d^n e^n \mid n \geq 0\}$
 - 类似上页PPT的方案二和方案三

以下哪些机器装置识别语言的能力与基本图灵机等价

- | | |
|---|-----------|
| A | 非确定图灵机 |
| B | 多带图灵机 |
| C | 多道图灵机 |
| D | 双栈下推自动机 |
| E | 带状态中储存图灵机 |
| F | 半无穷带图灵机 |
| G | 下推自动机 |

提交

小结

- 图灵机简介、形式化定义
- 图灵机的表示能力：接受语言、整数运算
- 扩展图灵机
 - 状态具有有穷存储功能的TM、多道TM、多带TM、非确定型TM
- 受限的图灵机
 - 半无穷带、多栈机器

课程整体内容

